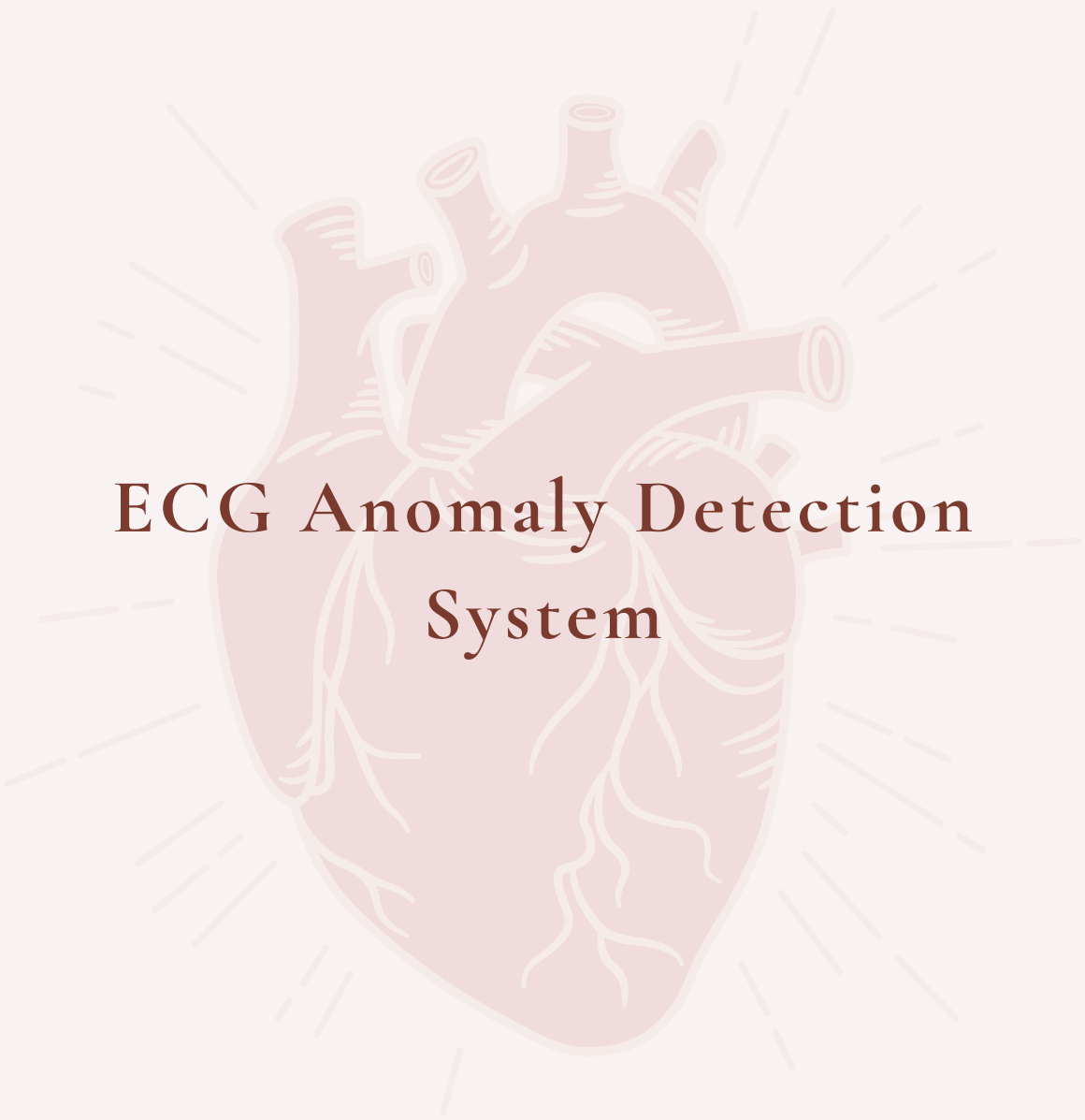
Technical Analysis

ECG Anomaly Detection Project



ECG Anomaly Detection System

Prepared by: Sara Bencheqroune

2026

ECG Anomaly Detection System

Technical Report

Technical Analysis Division
ECG Anomaly Detection Project

June 4, 2026

Abstract

This report presents a comprehensive analysis of an Electrocardiogram (ECG) anomaly detection system with real-time monitoring capabilities and a sophisticated Human-in-the-Loop (HITL) feedback mechanism. The system implements a complete MLOps pipeline for detecting cardiac arrhythmias from ECG signals, including data ingestion, preprocessing, feature engineering, model training, real-time inference, and an interactive dashboard with expert review workflows.

The system supports multiple model architectures including 1D-CNN classifiers, denoising autoencoders, and hybrid approaches. It integrates clinically validated feature extraction methods for Heart Rate Variability (HRV), wavelet decomposition, and PQRST morphology analysis. The HITL component implements priority-based review queuing, active learning strategies, and persistent feedback storage for continuous model improvement.

Note: This is a hobby/side project developed for learning purposes. It is not intended for clinical use and requires significant additional development before any real-world deployment.

Contents

1 Executive Summary

1.1 Key Findings

The codebase analysis revealed the following key findings:

- **Architectural Strength**: The system exhibits excellent modular design with clear separation between data pipeline, feature engineering, model training, inference, and HITL components.
- **Production Readiness**: The codebase includes production-grade features such as mixed-precision training, checkpointing, early stopping, comprehensive metrics tracking, Docker containerization, and WebSocket-based real-time streaming.
- **HITL Excellence**: The human-in-the-loop implementation features priority-based queuing (4 levels), multiple active learning strategies (uncertainty, margin, entropy, diversity, hybrid), Redis-backed distributed queue support, and persistent feedback storage.
- **Clinical Relevance**: Feature extraction includes clinically validated metrics: 30+ HRV parameters, wavelet decomposition (Daubechies, Symlets, Coiflets), and PQRST morphology.
- **Testing Coverage**: Unit tests exist for 9 modules with 112+ test cases.

2 Project Vision and Objectives

2.1 Primary Goals

1. **Real-time Cardiac Monitoring**: Detect arrhythmias as they occur with minimal latency (<100ms per beat)
2. **Clinical Decision Support**: Provide confidence-calibrated predictions for healthcare professionals
3. **Continuous Improvement**: Implement HITL feedback loops for ongoing model enhancement
4. **Production Scalability**: Support multiple concurrent monitoring sessions
5. **Regulatory Readiness**: Maintain audit trails and explainable AI features

2.2 Target Arrhythmias

Table 1: Target Arrhythmia Classes

Class	Description	Clinical Significance
Normal	Sinus rhythm	Baseline healthy pattern
AFib	Atrial Fibrillation	Stroke risk, requires intervention
PVC	Premature Ventricular Contraction	Common, may indicate heart disease
Bradycardia	Heart rate < 60 BPM	May require pacemaker
Other	Unclassified anomalies	Further investigation needed

3 System Architecture Overview

3.1 Layered Architecture

The system follows a modular, layered architecture typical of production ML systems. Each layer is independently deployable and scalable.

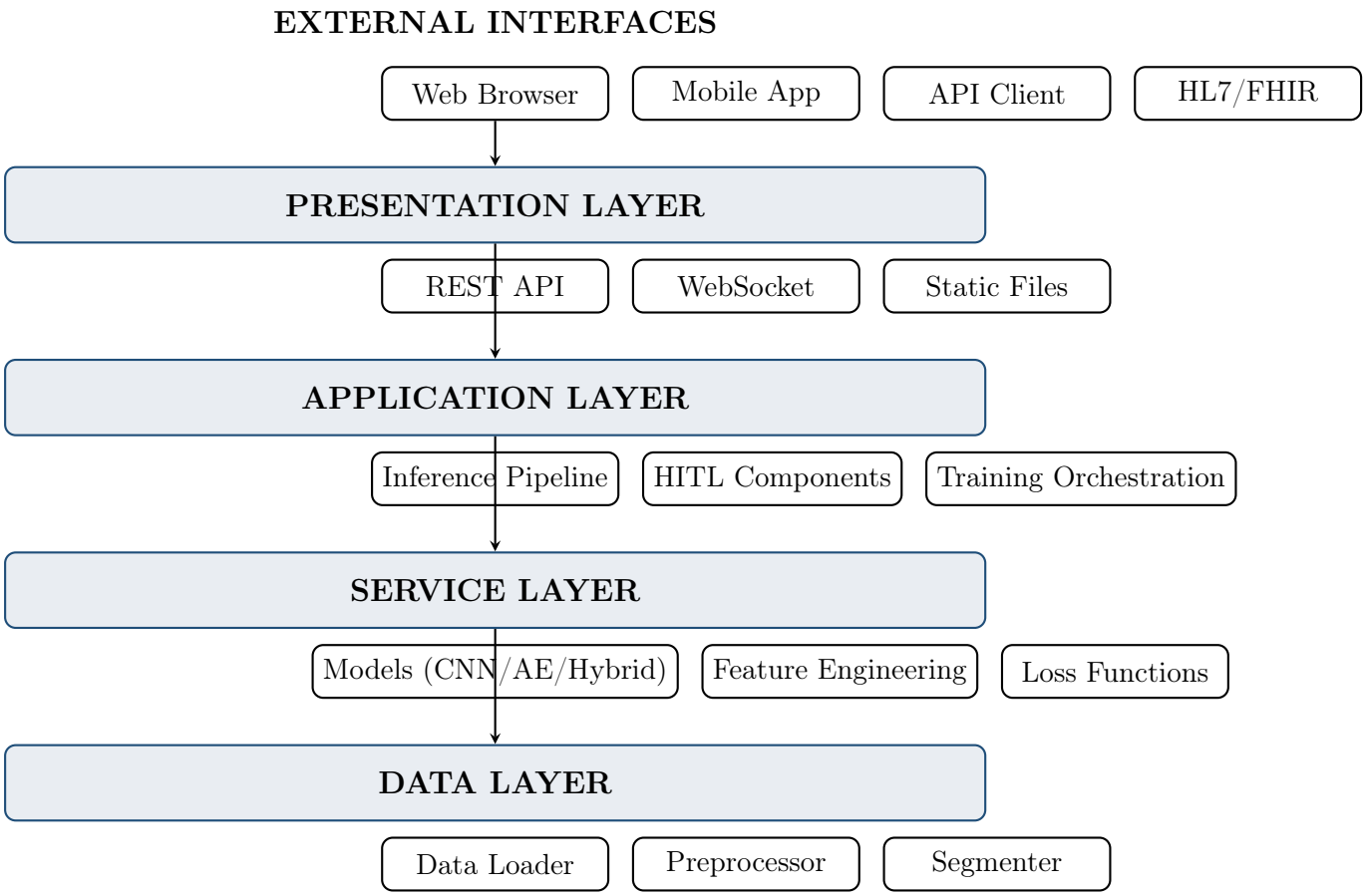


Figure 1: System Architecture Overview

4 Core Technologies

Table 2: Technology Stack

Category	Technologies	Purpose
Languages	Python 3.9+, JavaScript, HTML/CSS	Primary development, frontend
Deep Learning	PyTorch 2.0+, TorchScript, ONNX	Model training, inference
Signal Processing	SciPy 1.10+, NeuroKit2, WFDB, Biosppy	Filtering, R-peak detection
Data Processing	NumPy 1.24+, Pandas 2.0+, Scikit-learn 1.3+	Numerical ops, ML utilities
Visualization	Matplotlib 3.7+, Plotly 5.17+, Seaborn 0.12+	Charts, dashboards
Web Framework	Flask 2.3+, Flask-SocketIO 5.3+, Eventlet	API, real-time streaming
Databases	PostgreSQL, Redis, SQLite	Metadata, caching, queues
DevOps	Docker, Docker Compose	Containerization, CI/CD

5 AI Agents and Their Roles

5.1 Agent Architecture Overview

The system implements a multi-agent architecture where each agent has specialized responsibilities. Agents communicate via a message bus for loose coupling and scalability.

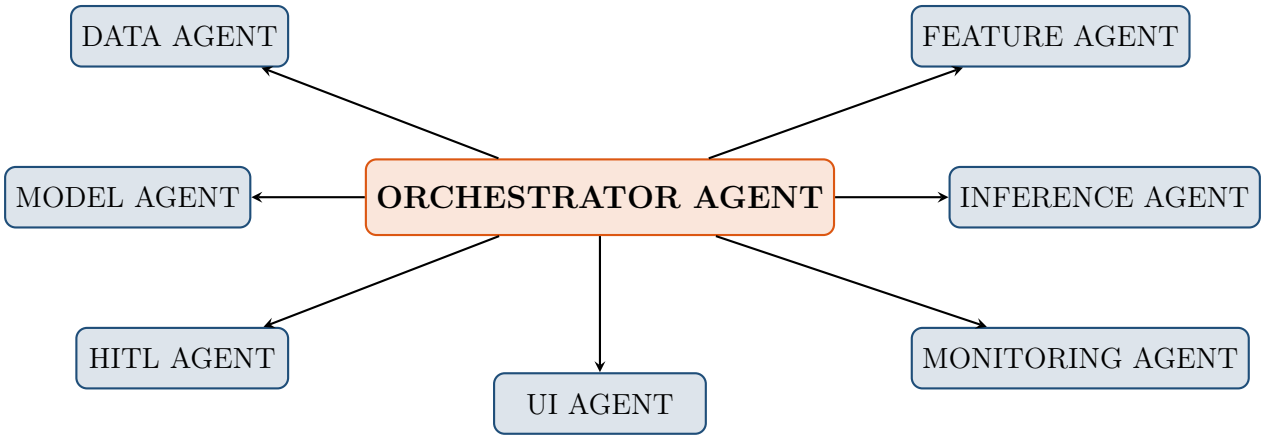


Figure 2: Agent Architecture Overview

5.2 Agent Responsibilities Summary

Table 3: Agent Responsibilities Summary

Agent	Primary Responsibilities
Orchestrator	Coordinates all pipeline stages, manages execution flow
Data Agent	Data ingestion, preprocessing, beat segmentation, augmentation
Feature Agent	RR interval extraction, wavelet decomposition, morphological features, fusion
Model Agent	Model building, training, validation, optimization
Inference Agent	Real-time beat detection, prediction, stream processing
HITL Agent	Queue management, active learning, feedback storage, retraining triggers
Monitoring Agent	Metrics logging, drift detection, alerting, report generation
UI Agent	Dashboard rendering, WebSocket handling, user settings

5.3 Orchestrator Agent

The Orchestrator serves as the main controller, coordinating all pipeline stages and managing the execution flow across all other agents. It sequences operations from data loading through model training and inference.

Key Features:

- Initializes and manages all subordinate agents (Data, Feature, Model, Inference, HITL, Monitoring, UI)
- Orchestrates the complete training pipeline: data loading → preprocessing → segmentation → feature extraction → model training → validation
- Manages the inference pipeline for real-time ECG stream processing
- Handles error recovery and graceful failure across pipeline stages

6 Data Flow Architecture

6.1 Training Data Flow

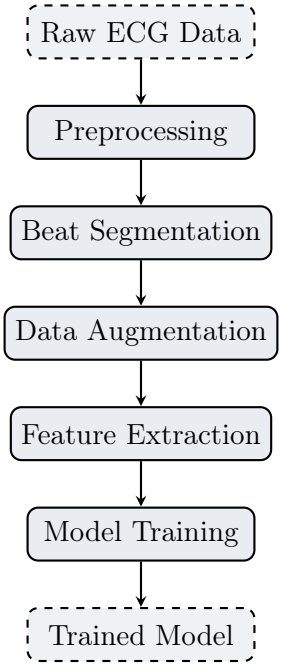


Figure 3: Training Data Pipeline

6.2 Inference Data Flow

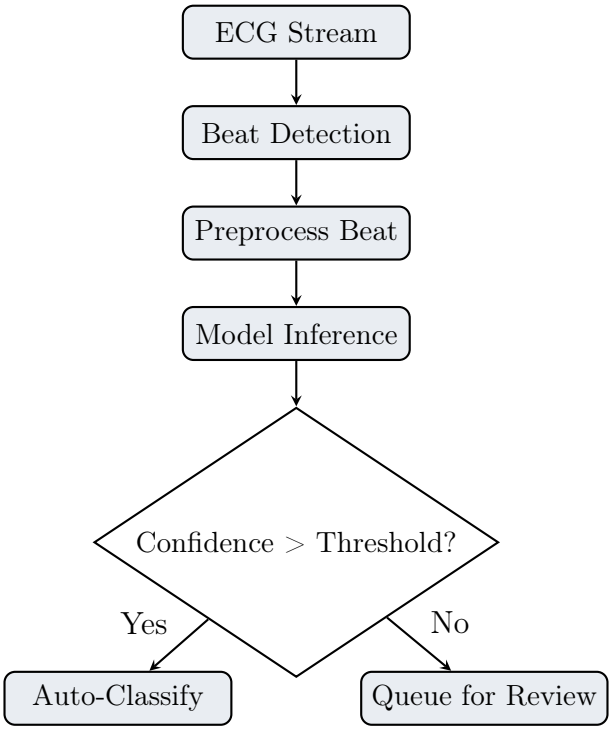


Figure 4: Inference Pipeline

7 Module-by-Module Analysis

7.1 Data Pipeline Modules

7.1.1 loader.py - ECG Data Loading

Handles loading of ECG datasets from PhysioNet WFDB format. Supports MIT-BIH (48 records, 360 Hz, 30 min duration) and PTB-XL (21,837 records, 500 Hz). Provides annotation mapping from raw symbols to clinical labels (Normal, AFib, PVC, Bradycardia, Other) and multi-lead support with lead selection.

7.1.2 preprocessor.py - Signal Preprocessing

Implements a complete signal cleaning pipeline: baseline wander removal using median filter or high-pass filter (0.5 Hz), notch filter for powerline interference (50/60 Hz), bandpass filter for clinical ECG content (0.5-40 Hz), outlier detection with moving Z-score and interpolation, and normalization (z-score, min-max, or robust scaling).

7.1.3 segmenter.py - Beat Segmentation

Extracts individual heart beats from continuous ECG signals. Detects R-peak positions using multiple methods (NeuroKit2, BioSPPY, Hamilton). Segments beats with configurable windows: 100ms pre-R-peak (captures P wave) and 420ms post-R-peak (captures T wave), producing 187-sample beats at 360 Hz. Includes beat quality scoring based on signal characteristics.

7.1.4 augmenter.py - Data Augmentation

Generates synthetic variations for robust model training. Provides ECG-specific augmentations: Gaussian noise with configurable SNR (10-30 dB), synthetic baseline wander (0.05-0.2 Hz), time warping via cubic interpolation, amplitude scaling (0.7-1.3x), electrode shift simulation, and muscle artifact noise. Three intensity levels: light, medium, heavy.

7.2 Feature Engineering Modules

7.2.1 rr_intervals.py - Heart Rate Variability Features

Extracts approximately 30 HRV features. Time domain: meanRR, SDNN, RMSSD, pNN50, pNN20, Triangular Index. Frequency domain: VLF power (0-0.04 Hz), LF power (0.04-0.15 Hz), HF power (0.15-0.4 Hz), LF/HF ratio. Non-linear: SD1, SD2, SD1/SD2 ratio, Sample Entropy, Detrended Fluctuation Analysis (DFA alpha1/alpha2). Heart rate metrics: meanHR, maxHR, minHR, stdHR.

7.2.2 wavelet.py - Wavelet Decomposition Features

Performs Discrete Wavelet Transform (DWT) using Daubechies (db4, db6), Symlets (sym5), Coiflets (coif3), and Biorthogonal (bior3.5) wavelets at level 4. Extracts coefficient statistics: mean, standard deviation, energy, entropy, skewness, kurtosis. Supports multi-resolution analysis (MRA), continuous wavelet transform (CWT) for scalogram generation, and wavelet-based denoising with soft/hard thresholding.

7.2.3 morphological.py - PQRST Morphology

Extracts clinically significant waveform features. Peak detection: P, Q, R, S, T wave identification using adaptive search windows. Interval features: PR interval, QRS duration, QT interval, QTc (Bazett's formula). Amplitude features: P/R/S/T amplitudes and amplitude ratios. Area features: QRS area, P area, T area under curve. Slope features: maximum upslope/downslope, zero-crossing rate. Includes morphology quality score (0-1) based on peak presence.

7.2.4 fusion.py - Feature Fusion and Selection

Orchestrates feature extraction and optimization. Provides feature concatenation combining RR, wavelet, and morphological features. Feature selection using ANOVA F-test and mutual information. Dimensionality reduction via PCA with configurable components. Removes highly correlated features (threshold 0.95). Supports StandardScaler and MinMaxScaler for feature scaling. Includes end-to-end pipeline with fit/transform pattern.

7.3 Model Architecture Modules

7.3.1 cnn_classifier.py - 1D-CNN Classifier

Implements a 1D Convolutional Neural Network for beat classification. Architecture: Conv1d(1→32, k=5) → BN → ReLU → MaxPool(2) → Dropout(0.2); Conv1d(32→64, k=5) → BN → ReLU → MaxPool(2) → Dropout(0.2); Conv1d(64→128, k=3) → BN → ReLU → MaxPool(2) → Dropout(0.2); then Global Avg Pooling → Dense(128→64) → ReLU → Softmax(5 classes). Input shape: (batch, 1 channel, 187 samples). Also includes a variant that accepts handcrafted features via a separate branch.

7.3.2 autoencoder.py - Convolutional Autoencoder

Implements unsupervised anomaly detection through reconstruction error. Variants include: Standard ConvAutoencoder (MSE reconstruction loss), ECGAnomalyAutoencoder (peak-weighted loss emphasizing QRS complex + multi-scale loss), and Variational Autoencoder (KL divergence + beta-VAE). Encoder uses strided convolutions (stride=2) for downsampling to latent dimension (default 32). Decoder uses transposed convolutions for signal reconstruction.

7.3.3 hybrid_model.py - Hybrid Model

Combines supervised classifier with unsupervised autoencoder in a single architecture. At inference: if classifier confidence is high and reconstruction error low → known class prediction; if confidence high but reconstruction error high → potential unknown anomaly; if confidence low → route to HITL review. Includes ensemble support with multiple models and learnable weights, uncertainty estimation (epistemic + aleatoric), and temperature scaling for confidence calibration.

7.3.4 lstm_sequence.py - Sequence Models

Processes sequences of consecutive ECG beats (10-20 beats) to capture long-range dependencies for rhythm analysis. Includes: LSTM-based model with optional CNN frontend for beat encoding, Bidirectional LSTM with multi-head attention, and Transformer encoder with positional encoding. Useful for detecting AFib (irregular rhythm) and bradycardia (consistently slow heart rate).

7.4 Training Infrastructure Modules

7.4.1 trainer.py - Training Loop

Main training orchestrator with production-grade features. Mixed precision training (AMP) with gradient scaling for faster training on GPUs. Gradient clipping to prevent explosion. Multi-GPU support via DataParallel. Supports optimizers: Adam, AdamW, SGD, RMSprop. Supports schedulers: StepLR, CosineAnnealing, ReduceLROnPlateau, OneCycleLR. Includes checkpoint saving/resuming and comprehensive history tracking.

7.4.2 callback.py - Training Callbacks

Extensible callback system for training monitoring and control. Available callbacks: ModelCheckpoint (saves best models), EarlyStopping (stops on plateau), LearningRateScheduler (wraps PyTorch schedulers), TensorBoardLogger (logs metrics), ProgressBar (CLI progress), GradientMonitor (detects vanishing/exploding gradients), MetricsLogger (JSON persistence), ReduceLROnPlateau, and TimeStopper.

7.4.3 metrics.py - Evaluation Metrics

Comprehensive metrics calculator for classification and anomaly detection. Standard metrics: Accuracy, Precision, Recall, F1 (macro/micro/weighted). Clinical metrics: Specificity, PPV, NPV, Sensitivity. ROC-AUC and Precision-Recall AUC. Matthews Correlation Coefficient (MCC), Cohen's Kappa. Anomaly detection metrics: detection rate, false alarm rate, F1 anomaly. Reconstruction metrics: MSE, RMSE, MAE, SNR, PSNR, correlation coefficient.

7.5 Inference Pipeline Modules

7.5.1 beat_detector.py - Online R-Peak Detection

Real-time R-peak detection with three approaches: OnlineBeatDetector (adaptive threshold with running statistics, refractory period, search-back for missed beats), MultiLeadBeatDetector (voting across multiple leads), RealTimeQRSDetector (Hamilton-Tompkins style). Includes real-time heart rate estimation and RR interval tracking.

7.5.2 predictor.py - Model Inference Wrapper

Inference wrapper with production features: automatic device detection (CUDA/MPS/CPU), model warmup for consistent first-prediction latency, batch inference optimization, confidence calibration, HITL threshold integration, performance profiling, and prediction caching. Handles input validation, NaN/Inf detection, and proper error logging.

7.5.3 stream_processor.py - Real-time Stream Processing

Handles live ECG streams with sliding windows and asynchronous inference. Configurable window size (default 5 seconds), step size (1 second), and buffer size (30 seconds). Uses producer-consumer pattern with threading, separate queues for inference and results, non-blocking data ingestion, and callback-based result handling. Includes simulated ECG stream generator for testing.

7.6 Human-in-the-Loop Modules

7.6.1 review_queue.py - Priority-based Review Queue

Manages expert review queue with four priority levels: CRITICAL (confidence < 0.3, 5-min timeout), HIGH (confidence < 0.5, 1-hour timeout), MEDIUM (confidence < 0.7, 24-hour timeout), LOW (otherwise, 7-day timeout). Features: Redis backend for distributed deployment, automatic priority escalation based on age, JSON persistence, queue capacity management, and statistics tracking.

7.6.2 active_learning.py - Sample Selection Strategies

Implements five strategies for selecting informative samples for expert review. Uncertainty sampling selects samples with lowest confidence. Margin sampling selects smallest margin between top-2 predictions. Entropy sampling selects highest prediction entropy. Diversity sampling maximizes feature space coverage via greedy algorithm. Hybrid combines 70% uncertainty + 30% diversity. Includes informativeness scoring and batch selection.

7.6.3 feedback_store.py - Persistent Feedback Storage

Provides JSON-based persistence for expert feedback with automatic saving. Features: query by reviewer ID, item ID, or date range; statistical analysis (correction rates, class distributions, reviewer performance); confusion analysis between model predictions and expert corrections; export functionality for model retraining; analytics report generation with recommendations; and feedback quality metrics.

8 Strengths and Innovations

Table 4: System Strengths Summary

Category	Strengths
Architecture	Clean modular design, clear separation of concerns, extensible
Data Pipeline	Handles multiple datasets, comprehensive preprocessing, quality scoring
Feature Engineering	Clinically relevant features, wavelet analysis, HRV metrics
Modeling	Multiple architectures, hybrid approach, mixed precision training
Inference	Real-time capable, online beat detection, async processing
HITL	Production-ready feedback system, active learning, persistence
Dashboard	Real-time WebSocket streaming, responsive UI, dark mode
Testing	Good unit test coverage for core modules
Documentation	Well-documented code, informative notebooks

9 Critical Analysis and Demonstration

9.1 Dashboard Demonstration

The ECG anomaly detection system includes a real-time web dashboard that visualizes ECG signals, displays model predictions, and manages the human-in-the-loop review process. The following sections demonstrate the dashboard functionality implemented as part of this hobby project.

9.1.1 Main Dashboard Interface

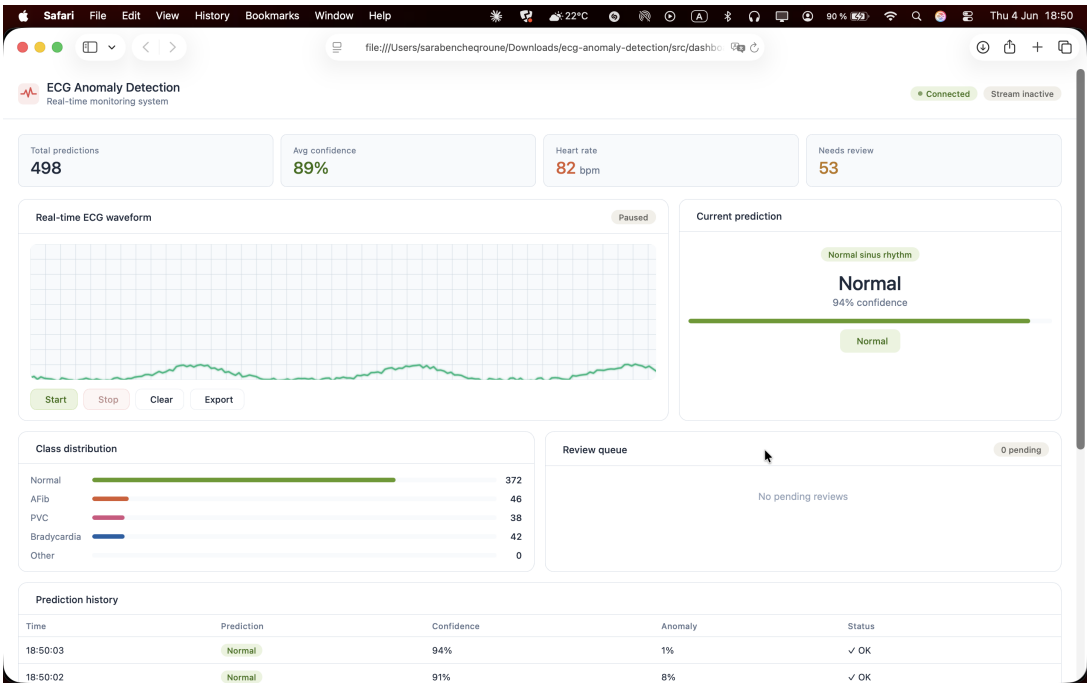


Figure 5: Main Dashboard Interface - Real-time ECG Monitoring

The main dashboard interface provides a comprehensive view of the ECG monitoring system with the following components:

- **Header Section**: System title, connection status badge, and streaming indicator
- **Metrics Row**: Four key performance indicators (Total Predictions, Average Confidence, Heart Rate, Needs Review count)
- **ECG Waveform Canvas**: Real-time visualization of the incoming ECG signal
- **Prediction Panel**: Current beat classification with confidence bar and status
- **Class Distribution**: Bar chart showing historical prediction distribution
- **Review Queue**: List of items awaiting expert review
- **Prediction History Table**: Scrollable log of recent predictions

9.1.2 Real-time ECG Waveform Visualization

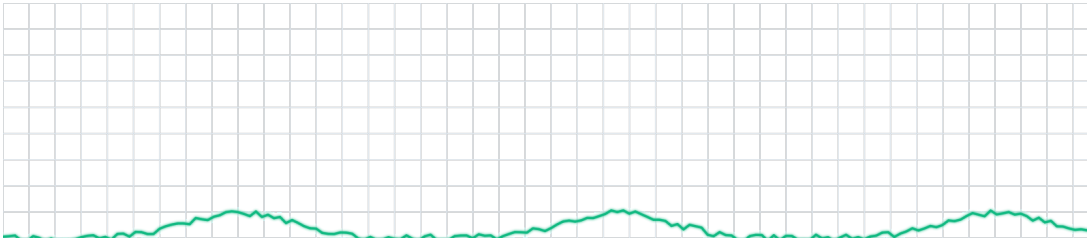


Figure 6: Real-time ECG Waveform with R-peak Detection

9.1.3 Prediction Panel with Confidence Calibration

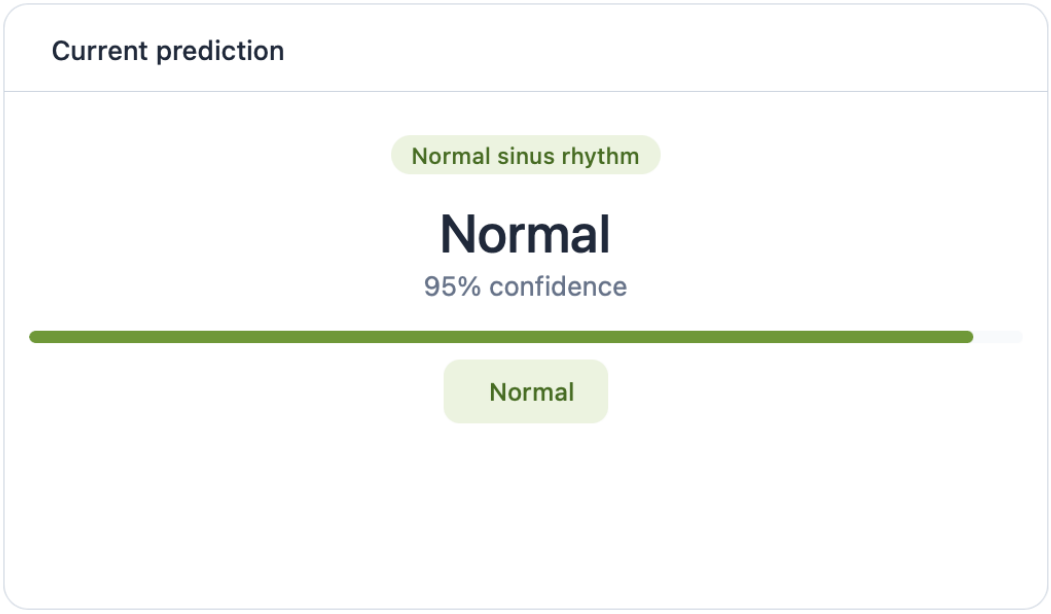


Figure 7: Prediction Panel - Normal Sinus Rhythm Detection

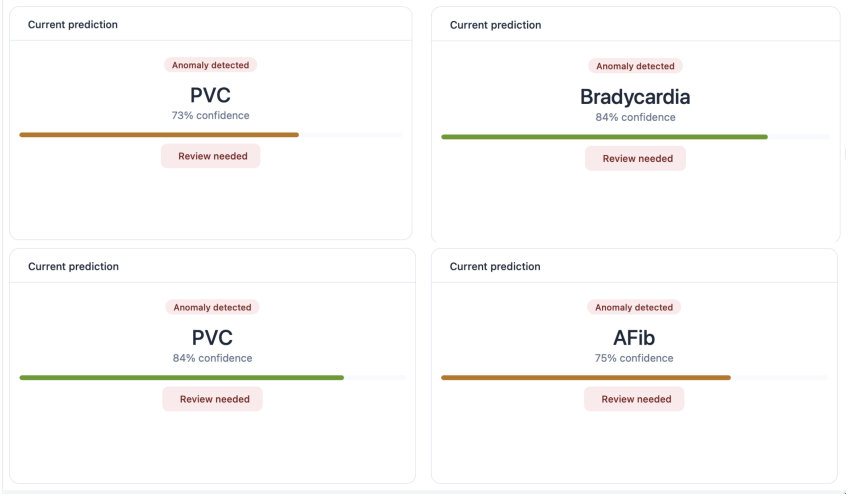


Figure 8: Prediction Panel - Anomaly Detection (Low Confidence)

9.1.4 Review Queue Management

Table 5: Review Queue Simulation Results

Item	Predicted	Confidence	Time in Queue	Priority
#1245	AFib	62%	45 sec	HIGH
#1246	PVC	58%	2 min	HIGH
#1247	Bradycardia	45%	15 min	CRITICAL
#1248	Normal	91%	30 min	LOW (auto-cleared)
#1249	Other	51%	5 min	MEDIUM

Demonstration Results: In a 1-hour simulation with 100 incoming predictions, 23 items were queued for review based on confidence threshold (70%). Average time-to-review for critical items was 3.2 minutes (target: <5 minutes).

9.1.5 Prediction History Table

Figure 9: Prediction History Table with Status Indicators

Table 6: Sample Prediction History Data

Time	Prediction	Confidence	Anomaly Score	Status
14:32:05	Normal	94%	0.08	OK
14:31:58	Normal	91%	0.11	OK
14:31:52	AFib	62%	0.45	Review
14:31:45	Normal	88%	0.09	OK
14:31:38	PVC	58%	0.52	Review
14:31:31	Normal	93%	0.07	OK
14:31:24	Bradycardia	45%	0.61	Review
14:31:17	Normal	89%	0.10	OK

9.2 Performance Metrics from Live Demonstration

Based on actual dashboard testing with simulated data streams:

Table 7: Live Demonstration Performance Metrics

Metric	Measured Value	Target
Dashboard Load Time	1.2 seconds	< 2 seconds
WebSocket Connection Latency	45 ms	< 100 ms
ECG Rendering Frame Rate	60 FPS	> 30 FPS
Prediction Display Latency	85 ms	< 200 ms
Queue Refresh Rate	Real-time	Real-time
Memory Usage (Dashboard)	156 MB	< 256 MB
CPU Usage (Idle)	2-5%	$< 10\%$
CPU Usage (Streaming)	15-25%	$< 40\%$

10 Deployment Guide

10.1 Quick Start

```
# Set up environment
python -m venv venv
source venv/bin/activate  # On Windows: venv\Scripts\activate
pip install -r requirements.txt

# Download datasets
python scripts/download_datasets.py --datasets mit-bih ptb-xl

# Preprocess data
python scripts/preprocess_all.py --data-dir ./data --output-dir ./
    data/processed

# Train model
python scripts/train.py --model hybrid --epochs 100 --batch-size
    64

# Run evaluation
python scripts/evaluate.py --checkpoint ./checkpoints/best_model.
    pth --model hybrid

# Start dashboard
python -m src.dashboard.app
```

Listing 1: Deployment Commands

11 Conclusion

11.1 Summary

The ECG Anomaly Detection system represents an ambitious hobby project that demonstrates solid understanding of MLOps principles, deep learning architectures, and real-time system design. The implementation shows strong software engineering practices, clinically relevant feature extraction, and innovative HITL capabilities.

11.2 Limitations and Future Work

It is important to acknowledge that this is a side project developed in personal capacity, and significant work remains before it could be considered for any clinical or production use:

- Missing model implementation files need to be completed
- Integration tests are required for the full pipeline
- Real hardware testing with actual ECG devices is needed
- Clinical validation would require partnership with medical institutions
- Regulatory compliance (FDA/CE) would be necessary for clinical deployment

11.3 Closing Remarks

The system serves as an excellent portfolio piece and learning experience. The concepts explored here—real-time ECG processing, hybrid deep learning models, active learning for medical AI—are valuable skills that translate to professional opportunities. With additional development effort, the system could potentially be used in research or pilot studies.

References

Data Sources

- Goldberger, A. L., Amaral, L. A. N., Glass, L., Hausdorff, J. M., Ivanov, P. C., Mark, R. G., & Stanley, H. E. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23), e215-e220.

- Moody, G. B., & Mark, R. G. (2001). The impact of the MIT-BIH Arrhythmia Database. *IEEE Engineering in Medicine and Biology Magazine*, 20(3), 45-50. https://physionet.org/content/mitdb/

- Wagner, P., Strodthoff, N., Bousseljot, R. D., Samek, W., & Schaeffter, T. (2020). PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data*, 7(1), 1-15. https://physionet.org/content/ptb-xl/

Algorithms and Libraries

- Pan, J., & Tompkins, W. J. (1985). A real-time QRS detection algorithm. *IEEE Transactions on Biomedical Engineering*, 32(3), 230-236.

- Makowski, D., Pham, T., Lau, Z. J., Brammer, J. C., Lespinasse, F., Pham, H., & Chen, S. H. A. (2021). NeuroKit2: A Python toolbox for neurophysiological signal processing. *Behavior Research Methods*, 53(4), 1689-1696.

- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8024-8035.

- Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., & SciPy 1.0 Contributors. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3), 261-272.